

From Single-Movie Matching to Profile-Based Movie Recommendations

Student: Polina Elatontseva **Team:** Individual

Email: redlymood@gmail.com **Date:** 2026-09-25

Assignment: Movie Recommender System — Week 2

Abstract

This work updates a small content-based movie recommender. The original app matched one movie with two others using Jaccard similarity. The new app compares a single active movie with a profile formed by averaging up to three movie vectors, and shows two top-five lists using cosine similarity. Data parsing, encoding, zero-score results and optional rating loading were corrected. A self-contained prompt produced a second implementation in a fresh AI session. Both implementations passed shared tests. On 93 eligible MovieLens users, the profile increased genre diversity from 0.059 to 0.173. Its long-tail share was 79.35%, versus 76.34% for one movie, but this advantage was not robust to tie-breaking. Relevance and catalogue coverage did not improve at the primary seed. The main lesson is that a richer profile can broaden a list, while discovery and retention require separate evidence.

Index Terms — content-based recommendation, cosine similarity, user profile, MovieLens, long tail, prompt engineering

1. Introduction

Problem statement and motivation

A user's taste can include more than one film. A single-seed recommender is useful for a current interest, but it may keep returning very similar titles. The assignment asks for cosine similarity, a multi-film profile, five recommendations, bug fixes and a comparison of the two approaches. It also asks whether the new approach can support catalogue discovery.

Concrete failure

The old parser omitted the leading `unknown` genre name. It shifted the labels and dropped the actual Western feature. *Unforgiven* (1992), a Western-only film, therefore had no genres. The app recommended *Toy Story* and *GoldenEye* with zero similarity. This failure was reproduced from the original Git revision, not inferred only from comments.

Contributions

- A corrected browser app with side-by-side single-film and profile top-five lists.
- A complete regeneration prompt and an independently generated, unedited implementation.
- A temporal offline comparison, long-tail analysis, automated checks and reproducible outputs.

2. Related Work

MovieLens 100K supplies 100,000 ratings by 943 users on 1,682 films; its documentation defines the 19 original genre flags [1]. Cosine similarity is the normalized dot product [2]. MDN documents explicit byte decoding, which is needed for the legacy movie titles [3]. Playwright provides reproducible browser interaction and screenshots [4].

3. Method

Features and user profile

Each film is represented by an 18-dimensional binary vector x . The original unknown flag is parsed but is not a preference feature. A profile is the arithmetic mean of the raw vectors of the selected unique movies:

$$p = (x_1 + x_2 + \dots + x_m) / m, \quad 1 \leq m \leq 3$$

A genre appearing in two of three films has weight $2/3$. The app does not normalize each film before averaging. Duplicate selections cannot give a film extra weight. A zero vector contains no known preference signal.

Scoring and ranking

$$\cos(q, x) = (q \cdot x) / (\|q\|_2 \|x\|_2)$$

Single Movie uses the active film as q . User Profile uses p . Both modes exclude every selected film, reject zero-score candidates, and return at most five results. Equal scores use the same fixed ID hash with seed 20260925. Popularity is never a tie-breaker. Empty lists are explained rather than padded with unrelated films.

Architecture and interaction

The existing HTML/CSS/JavaScript app was updated, with pure scoring functions extracted to `recommender.js`. Data parsing stays in `data.js`; `script.js` manages the history, active radio button and result panels. ES modules load over HTTP. All computations run in the browser, without model training or a backend. A history change clears stale results. Synchronous scoring replaces the unnecessary timer.

Original issue	Correction and evidence
19 flags matched to 18 names	Read fields 6–23 as real genres. Toy Story and Unforgiven have the correct features.
Zero scores and file-order ties	Filter scores ≤ 0 ; deterministic hash tie-break. Reversing the catalogue preserves the result.
Unused ratings blocked startup	Ratings now provide optional counts. A delayed or failed <code>u.data</code> request does not block recommendations.
Legacy bytes decoded as UTF-8	Strict UTF-8 with Windows-1252 fallback restores all 9 affected titles.
Status colors lost CSS specificity	Specific state selectors; browser checks confirm different success/error colors.

Prompt correction and regeneration

The new prompt specifies the exact schema, pure function contracts, formula, exclusions, tie rule, loading states and acceptance checks. A fresh OpenCode session received only this prompt and the raw data in an isolated directory. Its five output files are saved under `generated/`. Source and input hashes record provenance. The generated source was not manually corrected.

4. Experiments

4.1 Setup and leakage prevention

The global 80th-percentile timestamp is **1998-03-07 02:21:09 UTC**. Ratings strictly before it form training (79,999 rows); ratings at or after it form test (20,001 rows). Profiles and popularity counts use training only. A candidate must have known genres and a training rating: 1,614 films qualify.

A positive rating is 4 or 5. For each eligible user, the profile uses the last three positive training films; the single seed is the most recent of these. Timestamp ties use movie ID. All training-seen films, including low-rated films, are excluded from both methods. Test positives must be eligible and previously unseen. These requirements leave **93 of 943 users**, so the cohort is selective.

Three methods are compared with the same candidates, exclusions and $k=5$: corrected Jaccard-single, cosine-single and cosine-profile. The buggy original is only a software baseline; it is not mixed into the clean algorithm comparison.

4.2 Concrete top-five comparison

Selected history: **Toy Story, Star Wars and Unforgiven**. Active seed: **Unforgiven**. The table uses the full interactive catalogue, excluding all three seeds. Counts are full-sample UI metadata, unlike the training-only counts in the offline experiment.

#	Single Movie	User Profile
1	Tombstone (1993) cos=1.000; ratings=108	Return of the Jedi (1983) cos=0.745; ratings=507
2	Bad Girls (1994) cos=1.000; ratings=6	Kid in King Arthur's Court, A (1995) cos=0.680; ratings=22
3	Wild Bunch, The (1969) cos=1.000; ratings=43	Transformers: The Movie, The (1986) cos=0.680; ratings=32
4	Wyatt Earp (1994) cos=1.000; ratings=50	Empire Strikes Back, The (1980) cos=0.680; ratings=367
5	Dead Man (1995) cos=1.000; ratings=34	Super Mario Bros. (1993) cos=0.667; ratings=26

The single list stays within Westerns. The profile combines family animation, adventure, science fiction and Western interests; its strongest match is Return of the Jedi. It can find cross-genre titles such as Transformers, but can also move away from the user's current Western intent. A score from one query is not a calibrated preference probability.

Related-interest check

With Toy Story, Aladdin and Aladdin and the King of Thieves, both modes retain A Goofy Movie at rank 1. The profile promotes Hercules because the history adds a Musical signal. With one selected film, both modes produce identical lists and scores by construction; this is also tested.

4. Experiments (continued)

4.3 Metrics and primary results

Precision@5 is observed positive hits divided by five. Recall@5 divides by the user's eligible test positives. NDCG@5 uses binary relevance with a log2 rank discount. Diversity is the mean pairwise (1 – cosine) within a list. Coverage is distinct recommended films divided by the 1,614 eligible films. User metrics are macro-averaged.

The long tail is the bottom 80% of eligible films by training rating count: 1291 films after a 323-film head. Count ties use the primary hash seed. Long-tail share is the fraction of returned films in this group. All evaluated lists contain five films.

Metric	Jaccard single	Cosine single	Cosine profile
Precision@5	2.37%	2.37%	2.15%
Recall@5	1.05%	1.05%	0.72%
NDCG@5	0.0261	0.0261	0.0245
Long-tail share@5	76.34%	76.34%	79.35%
Catalogue coverage	13.88%	13.88%	12.95%
Within-list diversity	0.059	0.059	0.173
Mean training rating count	50.68	50.68	51.94
Mean genres per recommendation	2.00	2.00	2.77

At the primary seed, Jaccard and cosine produce identical ordered top-five lists for **93/93 users**. This is an empirical result for this coarse feature space, not a general equivalence theorem. Replacing the measure alone does not improve the measured ranking here.

4.4 Uncertainty and tie sensitivity

Paired bootstrap intervals use 2,000 resamples of the same 93 users. For profile minus single, the NDCG difference is -0.0015 (95% interval -0.017 to 0.014). The long-tail difference is 3.01 percentage points (interval -2.80 to 9.03). Both intervals include zero. The diversity difference is 0.114 (interval 0.076 to 0.152).

Five predeclared tie seeds were evaluated: 20260925, 7, 42, 123 and 2026. Single-mode tail share ranges from 76.34%–81.51%; profile-mode share ranges from 72.47%–79.35%. Profile NDCG ranges from 0.017 to 0.036. Coarse genre signatures leave many ties, so a single-seed result is not sufficient evidence of superiority.

Measured outcome: a more varied list, but no established relevance or long-tail improvement. Profile coverage is lower (209 distinct films versus 224), even though within-list diversity is higher.

5. Discussion

Why normalization helps with multi-genre bias

For binary movie vectors with a and b active genres and c shared genres, cosine equals $c / \sqrt{(ab)}$. Adding an unmatched genre increases the denominator without increasing the numerator. A film cannot gain a score advantage merely by carrying irrelevant extra genre tags.

Query $q = [1,1,0,0]$	Raw dot product	Cosine
Exact candidate $[1,1,0,0]$	2	1.000
Extra unmatched genres $[1,1,1,1]$	2	0.707

However, matching extra dimensions of a broad profile can genuinely raise similarity. The profile recommends 2.77 genres per film on average, compared with 2.00 in single mode. Normalization does not ban multi-genre films, remove genre-frequency bias or directly measure popularity. Also, raw-vector averaging lets a genre-rich history film introduce more feature directions. Normalizing each seed before averaging would be a different design, not the implementation tested here.

Jaccard also penalizes unmatched genres. Therefore, cosine's denominator alone is not evidence that it beats Jaccard. Its practical benefit in this assignment is a direct, understandable comparison between binary movie vectors and fractional profiles.

Catalogue discovery, fatigue and retention

At the primary seed the profile places slightly more long-tail films in each list, but the difference is uncertain and changes with tie-breaking. The average recommended film is not less popular: training counts are 51.94 versus 50.68. A tail share and a mean popularity measure different aspects of a distribution; a few popular films can raise the mean.

Item-to-item is better at preserving a narrow current intent and has higher catalogue coverage in this cohort. The averaged profile is better at within-list genre variety, but it may blur separate interests. Neither approach is a clear long-tail winner. A high tail share alone can reward obscure, irrelevant films, so it must be read together with relevance and coverage.

Variety may reduce repetitive recommendations and support return visits. That is a product hypothesis, not a measured retention effect. MovieLens contains historical ratings, but no exposure logs from these two new algorithms. A later controlled user study or A/B test should measure satisfaction, repeat visits and recommendation fatigue.

Limitations and next improvement

Only 93 users span the global cutoff and meet the inclusion criteria. Unrated films have unknown relevance, and rating counts reflect the MovieLens sample rather than box-office popularity. Eighteen genre features cannot represent plot, acting, tone or changing intent. The next useful experiment would add richer content features and compare multiple-interest profiles under the same temporal protocol.

6. Verification and AI Usage Disclosure

Verification performed in this session

Check	Observed result
Shared Node.js contract tests	12/12 pass across maintained and generated implementations.
Headless Chromium browser scenarios	12/12 pass: loading failures/delays, histories, zero matches, rapid clicks, safe text, keyboard and mobile width.
Cross-implementation comparison	All 1,682 single-film queries and 93 three-film history profiles match; score tolerance 10^{-12} .
Independent metric verification	Python recomputed all 279 user/method metric rows, aggregate means, exclusions and coverage.
Prompt provenance	Prompt and five generated source hashes match the generation manifest; no manual generated-source edits.

One test initially required the literal total “100,000” in a status message, although the prompt did not. It was corrected to wait for numeric card counts and verify unchanged errors/ranks. Full conformance also found trailing spaces in three raw titles; the maintained parser was aligned to the generated parser's trimming. These findings are recorded in `validation-notes.md`.

Tools and the role of AI

OpenCode with OpenAI `gpt-6-astra` was used for repository analysis, implementation, prompt design, test and experiment scripts, and report drafting. A separate fresh session generated the second app. Node.js 25.9.0, Playwright 1.63.0 / Chromium 153.0.8010.12, and Python 3.9 were used for execution and verification. The maintained code and generated code are both AI-assisted work.

The student specified and approved the requirements and the proposed comparison. Automated checks were executed by the assistant; this report does not claim a separate manual student browser test. Desktop/mobile screenshots were reviewed, but other browser engines and devices were not tested. One successful prompt run demonstrates feasibility, not a guarantee that every future generation will pass.

Session log and reproducibility

The main native session export is `elatontseva_a02_session.json`; the independent generation export is `generation_session.json`. They are local submission files. Generation ID: `ses_f2c7f6cfbffewv3CZaGzNy7EQk`. The repository stores the prompt, both implementations, data, tests, metrics, source hashes and this report's HTML/build script. See `RUNNING.md` for exact commands. Checks were executed on 2026-09-24; the requested report date is 2026-09-25.

References

1. GroupLens Research, “MovieLens 100K Dataset README.” files.grouplens.org/datasets/movielens/ml-100k/README. Dataset citation: F. M. Harper and J. A. Konstan, “The MovieLens Datasets: History and Context,” ACM TiiS, 2015, doi:10.1145/2827872.
2. scikit-learn, “cosine_similarity.” scikit-learn.org — cosine_similarity API documentation.
3. MDN Web Docs, “TextDecoder.” developer.mozilla.org/en-US/docs/Web/API/TextDecoder.
4. Microsoft, “Playwright Documentation.” playwright.dev/docs/intro. Browser automation used for the recorded verification.

Online documentation consulted 2026-09-24. Data are attributed to GroupLens Research, University of Minnesota.